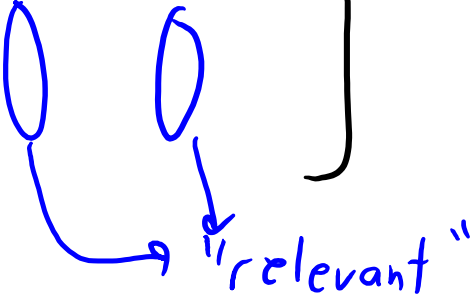


CPSC 340: Machine Learning and Data Mining

Regularization

Last Time: Feature Selection

- Last time we discussed **feature selection**:
 - Choosing set of “relevant” features.

$$X = \begin{bmatrix} \text{ } & \text{ } \end{bmatrix} \quad y = \begin{bmatrix} \text{ } \end{bmatrix}$$


- Most common approach is **search and score**:
 - Define “score” and “search” for features with best score.
- But it’s **hard to define the “score” and it’s hard to “search”**.
 - So we often use greedy methods like **forward selection**.
- Methods work ok on “toy” data, but are **frustrating on real data**.
 - Different methods may return very different results.
 - Defining whether a feature is “relevant” is complicated and ambiguous.

Last Time: Is “Relevance” Clearly Defined?

- Consider a supervised classification task:

Parent A	Parent B	Partner
1	0	0
0	1	1
0	0	1
1	1	1

Eye
0
0
0
1

- What are the “relevant” features for this problem?

Is “Relevance” Clearly Defined?

- What if we add grandparents?

Parent A	Parent B	Partner	Grandparent
1	0	0	1
0	1	1	1
0	0	1	0
1	1	1	1

Eye
0
0
0
1

- Is “Grandparent” relevant?
 - May be relevant, especially if “Parent A” or “Parent B” have non-blue eyes
 - But **irrelevant** if “Parent A” and “Parent B” both have blue eyes.
 - This is an example of **context specific relevance**.

Last Time: Is “Relevance” Clearly Defined?

- What if we only know about great-great-great-great-grandparents?

Ancestor 1	Ancestor 2	Partner
1	0	0
0	1	1
0	0	1
1	1	1

Eye
0
0
0
1

- A relevant feature may have a **tiny effect**.
- But there could be also **confounding factors**:
 - Maybe blues eyes are common where your family is from.

Is “Relevance” Clearly Defined?

- What if we don't know about parents, but know about siblings? :

Sibling 1	Sibling 2	Partner
1	1	0
0	1	1
1	0	1
0	1	1

Eye
1
0
0
0

- Sibling is “relevant” for prediction, but it is not the cause.
- “Relevance” for prediction does **not imply a causal relationship**.
 - Causality can even be reversed...

Last Time: Is “Relevance” Clearly Defined?

- What if we don't know about parents, but know about “baby”?

Partner	Baby
0	0
1	1
0	1
1	1

Eye
0
0
0
1

- Baby is relevant, although causality is reversed.
- Is partner now relevant?

Is “Relevance” Clearly Defined?

- Warnings about feature selection:
 - If features can be predicted from features, you can't know which to pick.
 - A feature is only “relevant” in the context of available features.
 - A “relevant” feature may have a tiny effect.
 - Confounding factors can change whether features are relevant.
 - “Relevance” for prediction does not imply a causal relationship.
 - “Relevance” may be conditional on values of certain features.

Is this hopeless?

- We often want to do feature selection we so have to try!
- Different methods are affected by problems in different ways.
- These “problems” don’t have right answers but have **wrong answers**:
 - **Variable dependence** (“Parent A” and “Parent A (2)” have same information).
 - But should take at least one.
 - **Conditional independence** (all “Grandparent” information is captured by parents)
 - Should take “Grandparent” only if “Parent” missing or has non-blue eyes.
- These “problems” have **application-specific answers**:
 - **Tiny effects**.
 - **Context-specific relevance** (is “partner” relevant if given “baby”?).
- See bonus slides for discussion of **causality and confounding** issues (like RCTs in medicine).
 - Unless you control data collection, **standard feature selection methods cannot address those issues**.

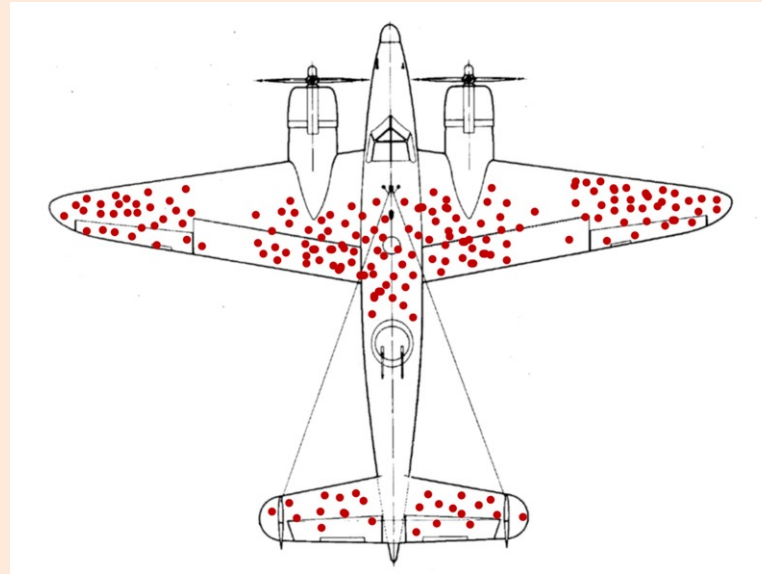
My advice if you want the “relevant” variables.

- Try the **association approach**.
- Try **forward selection with different values of λ** .
- Try out a few other feature selection methods too.
- **Discuss the results** with the domain expert.
 - They probably have an idea of why some variables might be relevant.
- **Do not be overconfident:**
 - These methods are probably not discovering how the world truly works.
 - “The algorithm has found that these variables are helpful in predicting y_i .”
 - Then a warning that these models are not perfect at finding relevant variables.

بسیار جالب

Related: Survivorship Bias

- Plotting location of bullet holes on planes returning from WW2:

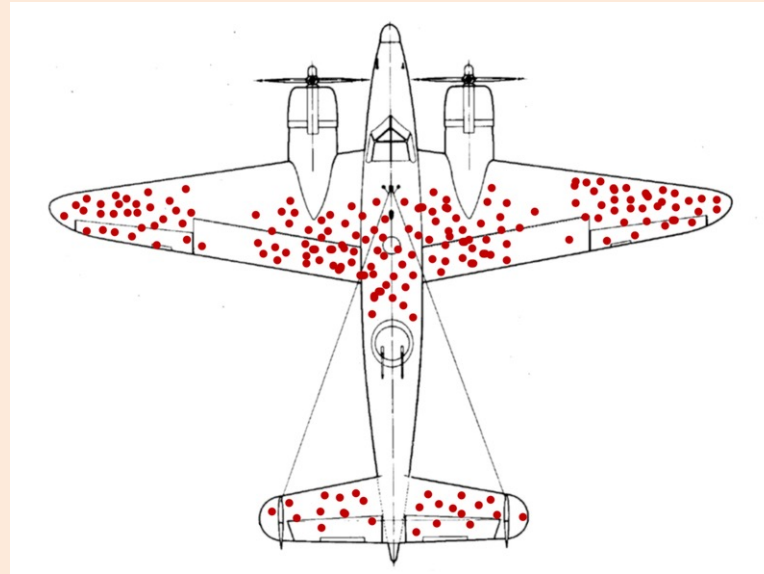


- Where are the “relevant” parts of the plane to protect?
 - “Relevant” parts are actually **where there are no bullets.**
 - **Planes shot in other places did not come back** (armor was needed).

selection
Bias

Related: Survivorship Bias

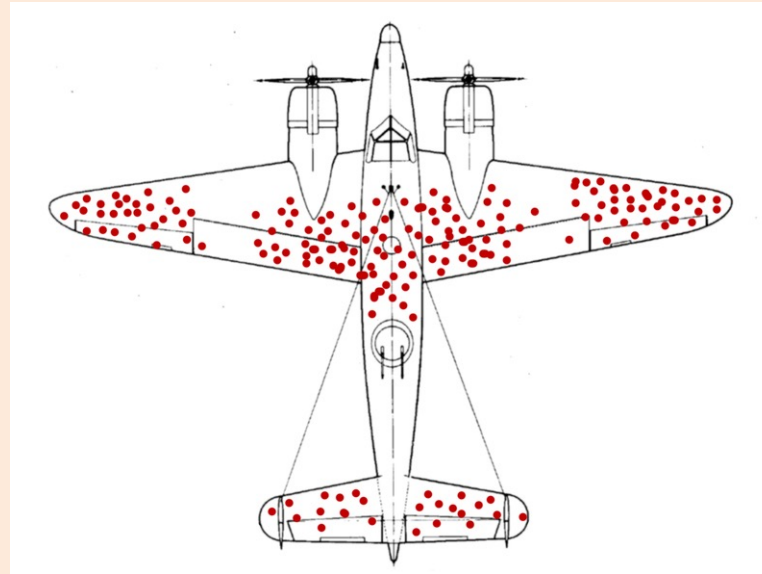
- Plotting location of bullet holes on planes returning from WW2:



- This is an example of “survivorship bias”:
 - Data is not IID because you only sample the “survivors”.
 - Causes havoc for feature selection, and ML methods in general.

Related: Survivorship Bias

- Plotting location of bullet holes on planes returning from WW2:



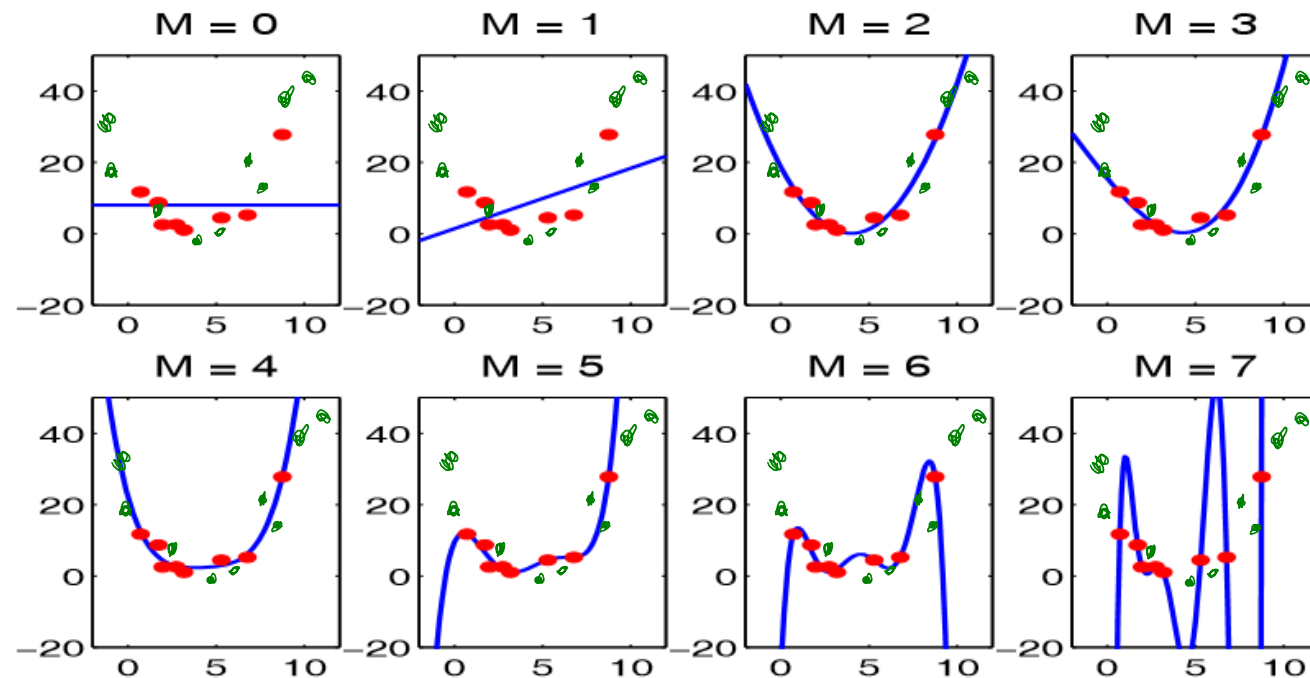
- People come to **wrong conclusions due to survivor bias** all the time.
 - Article on “secrets of success”, focusing on traits of successful people.
 - But ignoring the number of non-super-successful people with the same traits.
 - [Article](#) hypothesizing about various topics (allergies, mental illness, etc.).

Data selection
Bias

Next Topic: Regularization

Recall: Polynomial Degree and Training vs. Testing

- We've said that **complicated models tend to overfit more**.



- But what if we **need a complicated model**?

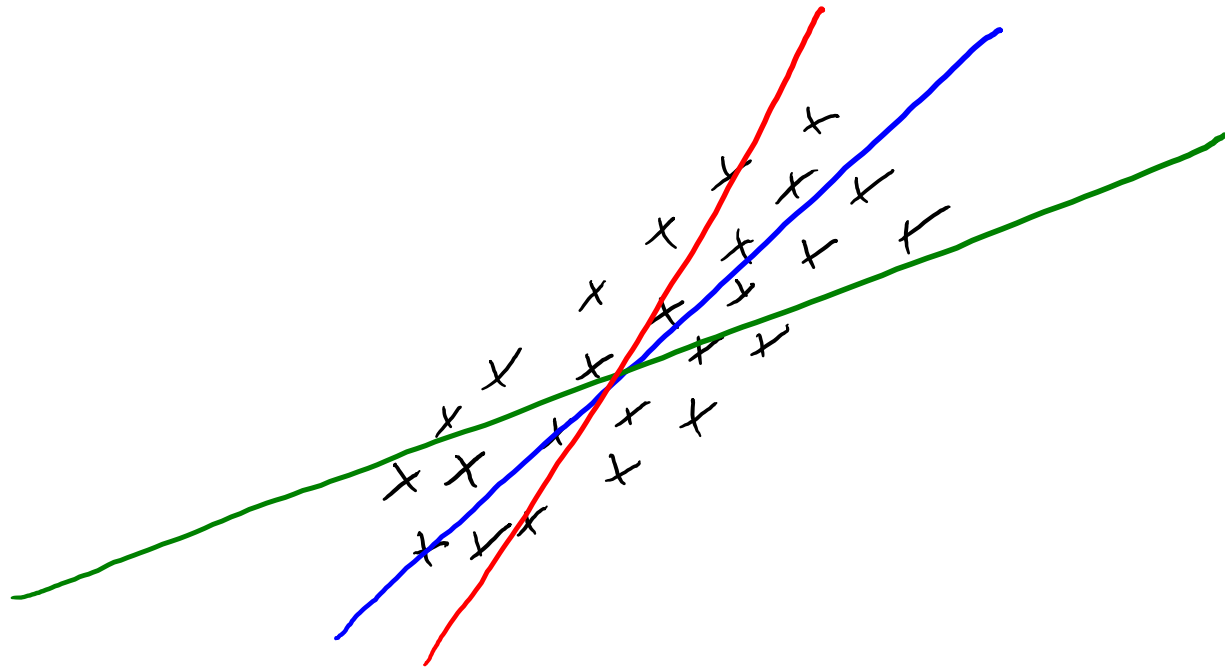
Controlling Complexity

- Usually “true” mapping from x_i to y_i is complex.
 - Might need high-degree polynomial.
 - Might need to combine many features, and do not know “relevant” ones.
- But complex models can overfit.
- So what do we do???
- Our main tools:
 - Model averaging: average over multiple models to decrease variance.
 - Regularization: add a penalty on the complexity of the model.

growing
bagging.

Would you rather?

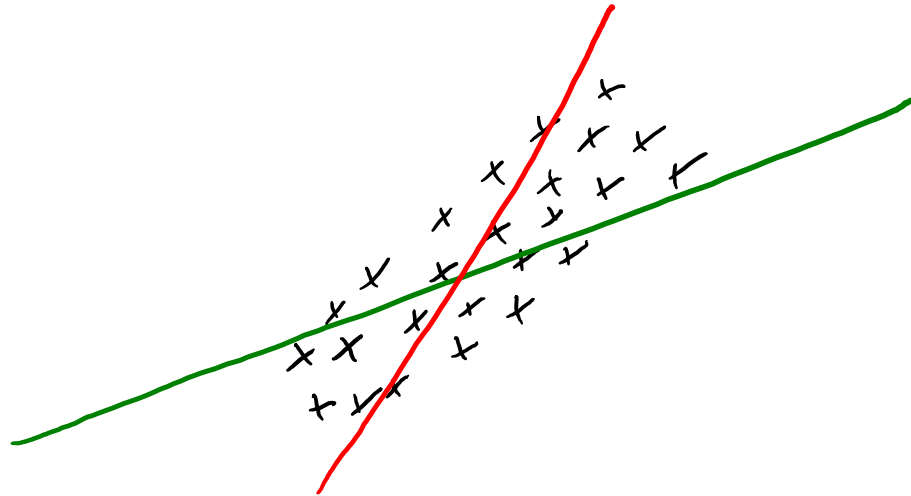
- Consider the following dataset and 3 linear regression models:



- Which line should we choose?

Would you rather?

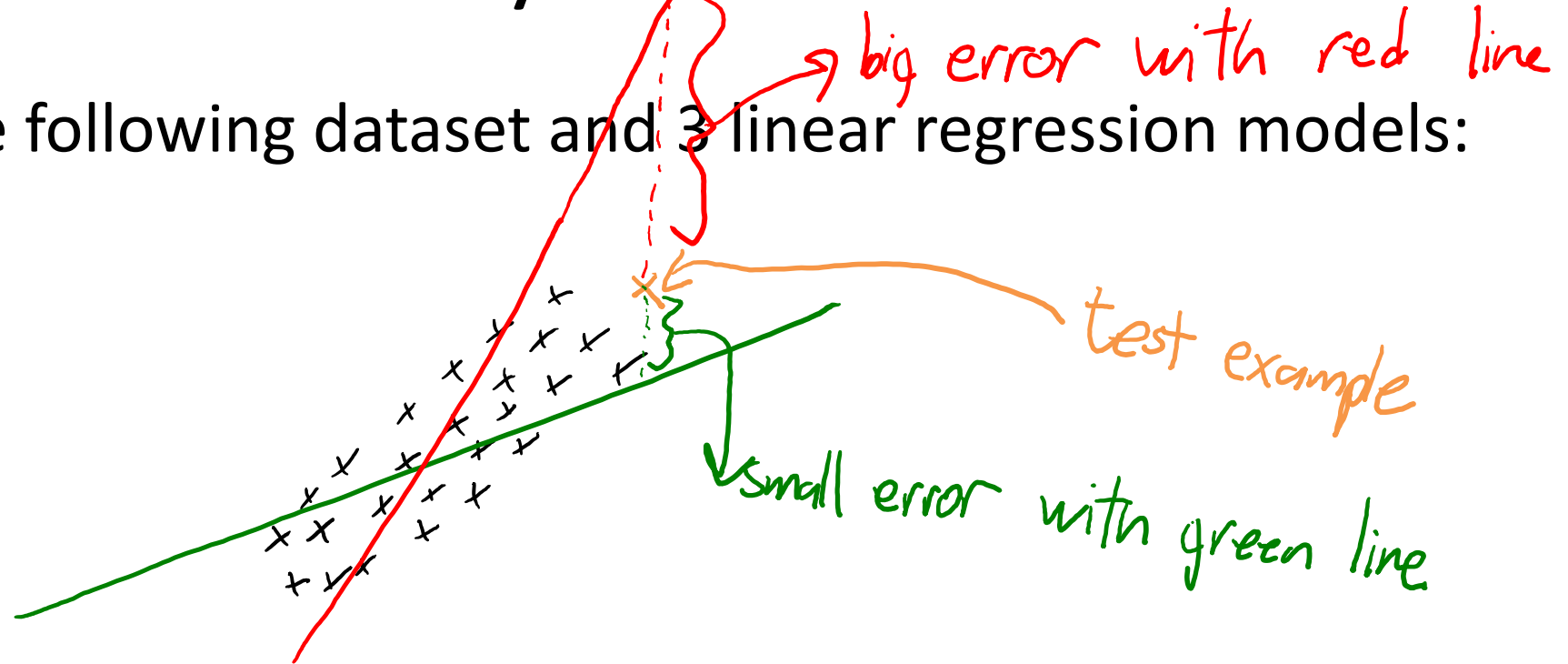
- Consider the following dataset and 3 linear regression models:



- What if you are forced to choose between **red** and **green**?
 - And assume they have the same training error.

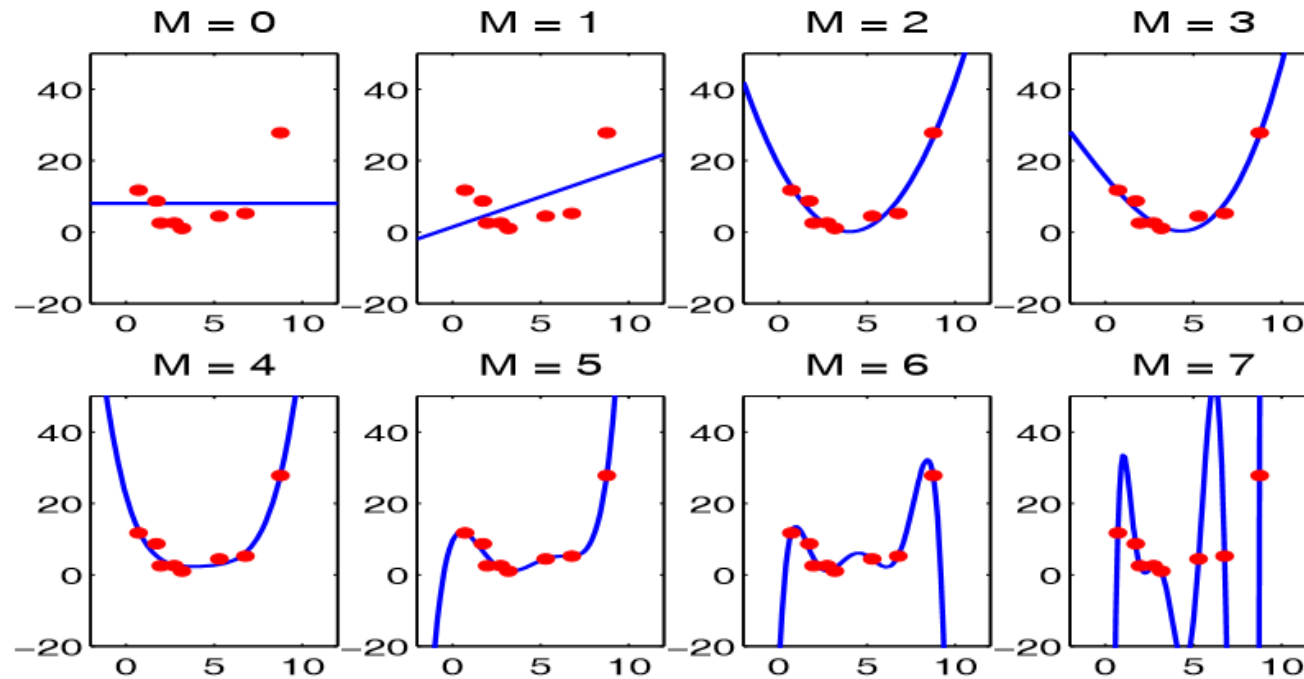
Would you rather?

- Consider the following dataset and 3 linear regression models:



- You should **pick green**.
 - Slope is smaller: **small change in x_i has a smaller change in prediction y_i** .
 - Green line's predictions are **less sensitive to having 'w' exactly right**.
 - Since green 'w' is less sensitive to data, test error might be lower.

Size of Regression Weights and Overfitting



- The regression weights w_j with degree-7 are huge in this example.
- The degree-7 polynomial would be less sensitive to the data, if we “regularized” the w_j so that they are small.

$$\hat{y}_i = 0.0001(x_i)^7 + 0.03(x_i)^3 + 3 \quad \text{vs.} \quad \hat{y}_i = 1000(x_i)^7 - 500(x_i)^6 + 890x_i$$

L2-Regularization

- Standard regularization strategy is L2-regularization:

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \quad \text{or} \quad f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

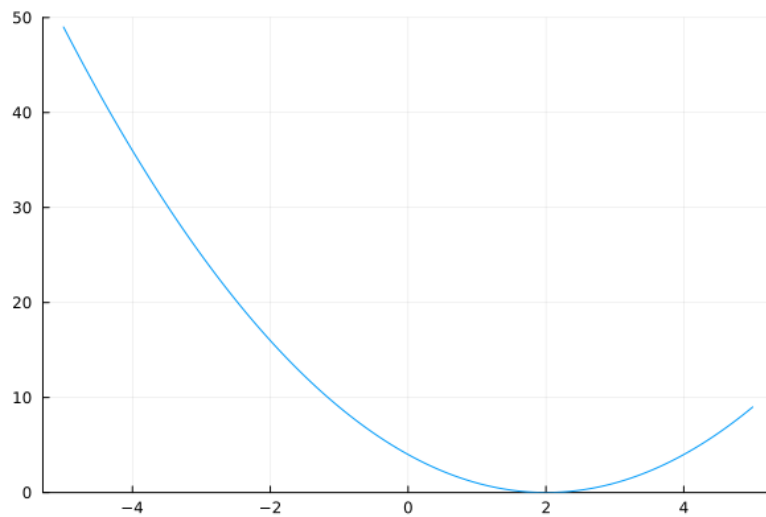
- For some regularization parameter $\lambda > 0$.
- Intuition: large slopes w_j tend to lead to overfitting.
- Objective balances getting low error vs. having small slopes ' w_j '.
 - “You can increase the training error if it makes ‘w’ much smaller.”
 - Nearly-always reduces overfitting.
- In terms of fundamental trade-off:
 - Regularization increases training error.
 - Regularization decreases generalization gap.

Regression
→ Back to mean

L2-Regularization

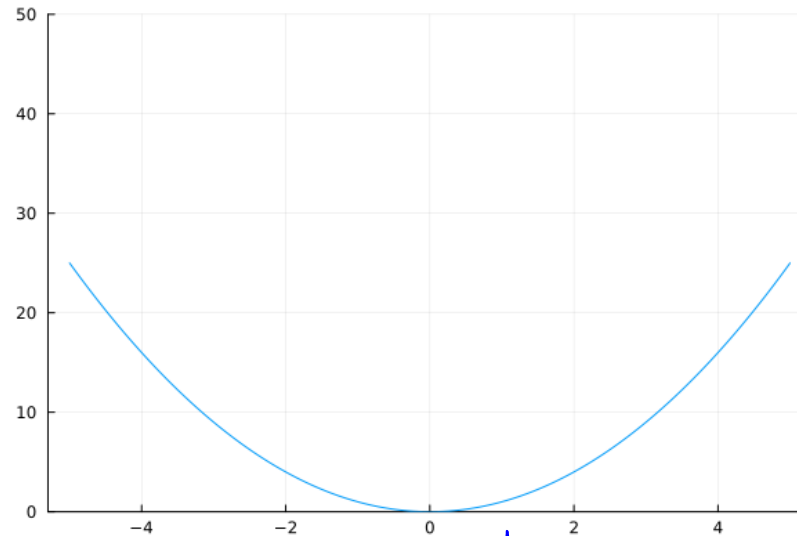
- Visualizing squared error with and without regularization (d=1):

Squared error (no regularization)



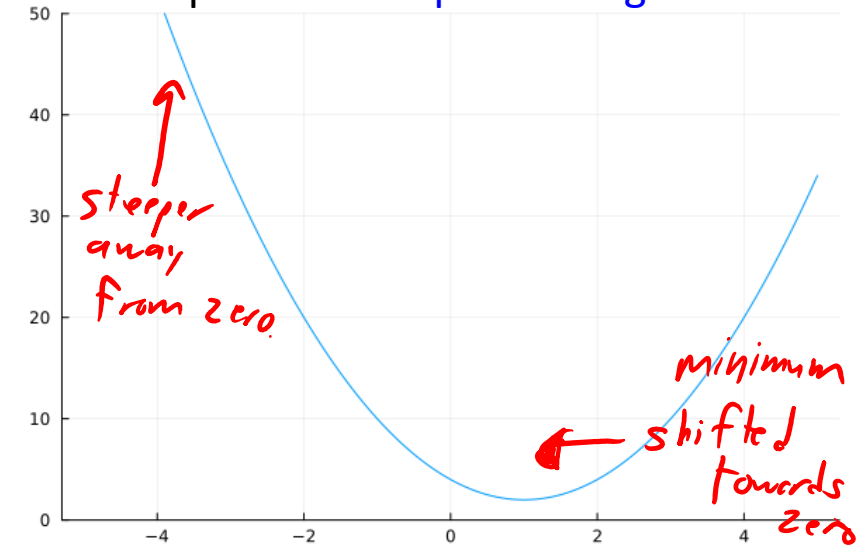
$$f(w) = \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2$$

L2-regularization function ($\lambda = 1$)



$$f(w) = \frac{\lambda}{2} \sum_{j=1}^d w_j^2$$

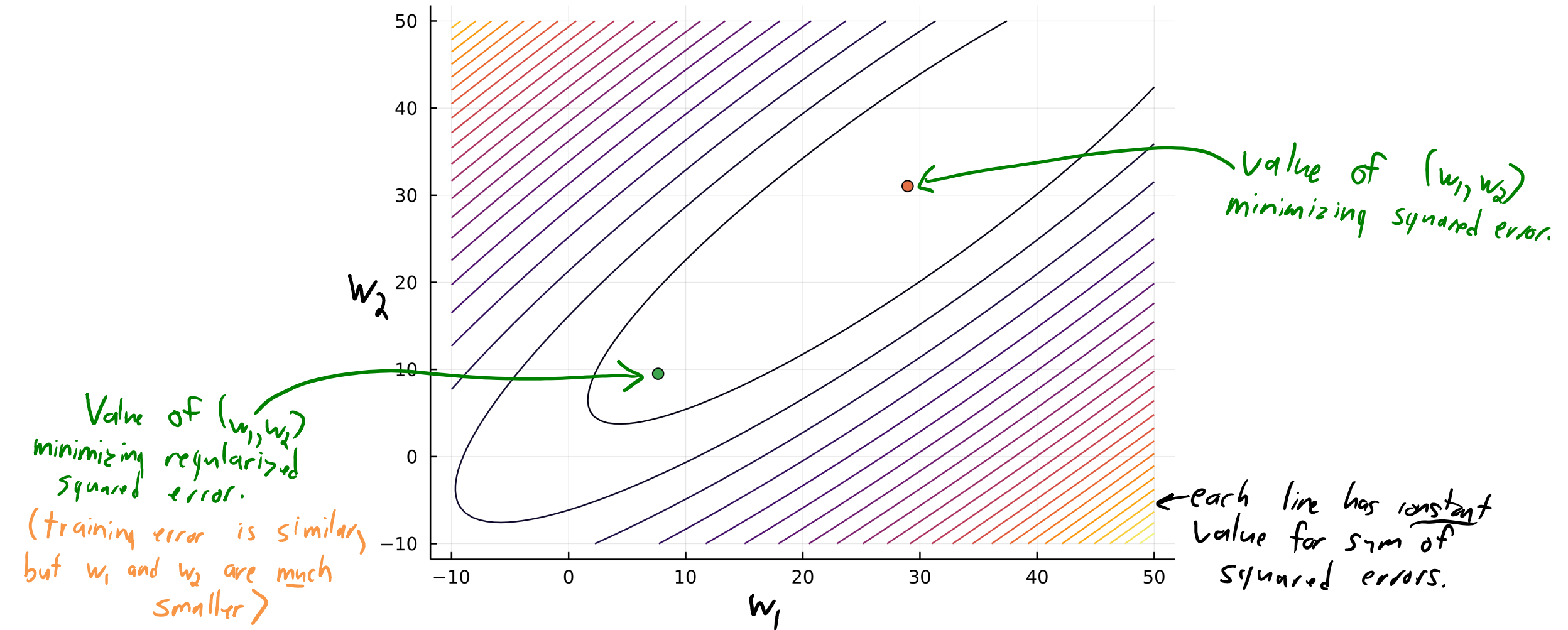
Squared error plus L2-regularization



$$f(w) = \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2$$

L2-Regularization

- Visualizing squared error as a function of parameters ($d=2$):



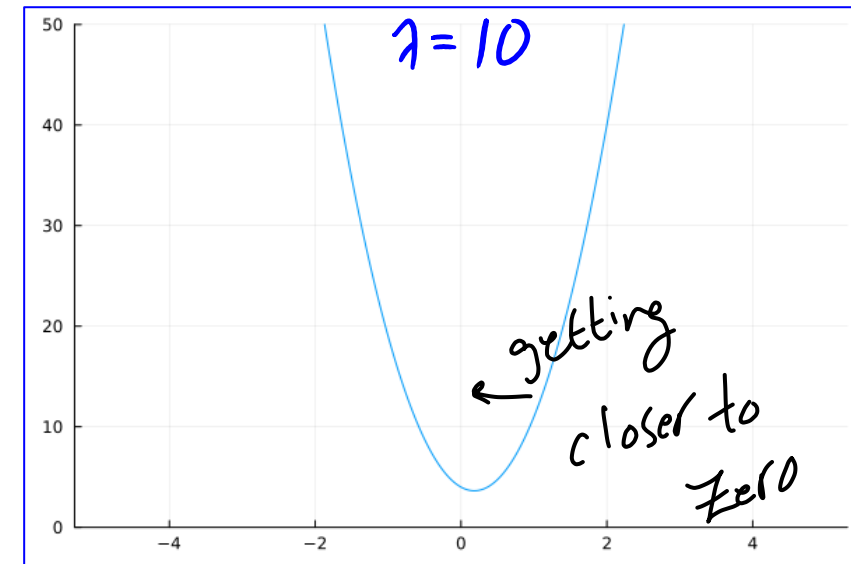
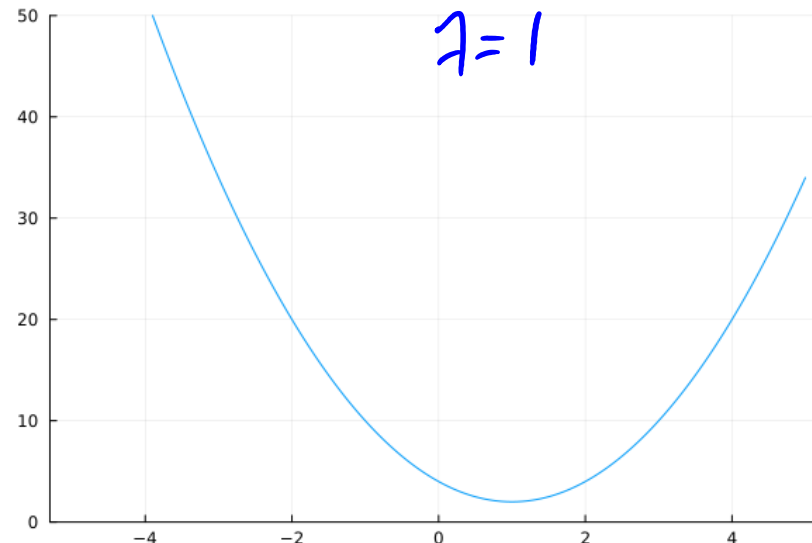
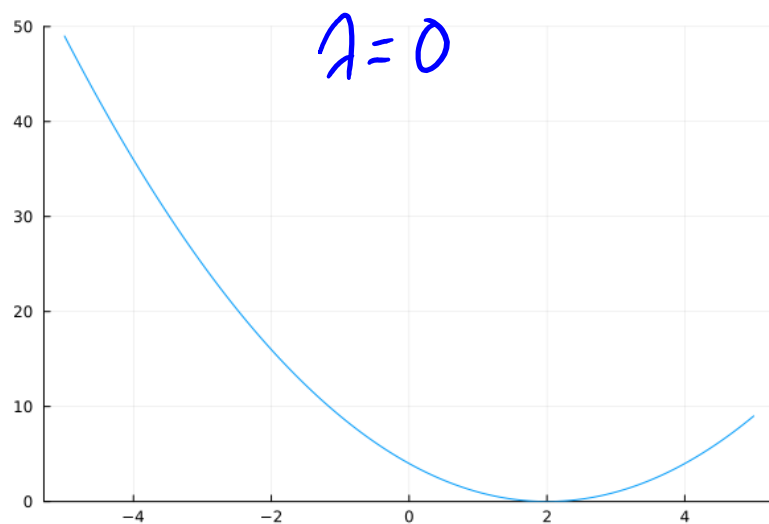
L2-Regularization

- L2-regularized least squares:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

– Regularization parameter $\lambda > 0$ controls “strength” of regularization.

- Large λ puts large penalty on slopes (worse training error, better generalization gap).



L2-Regularization

- L2-regularized least squares:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

- Regularization parameter $\lambda > 0$ controls “strength” of regularization.
 - Large λ puts large penalty on slopes (worse training error, better generalization gap).
- How should you choose λ ?
 - Theory: as ‘n’ grows λ should be in the range $O(1)$ to $(\sqrt{n})$.
 - Practice: optimize validation set or cross-validation error.
 - This almost always decreases the test error.

L2-Regularization “Shrinking” Example

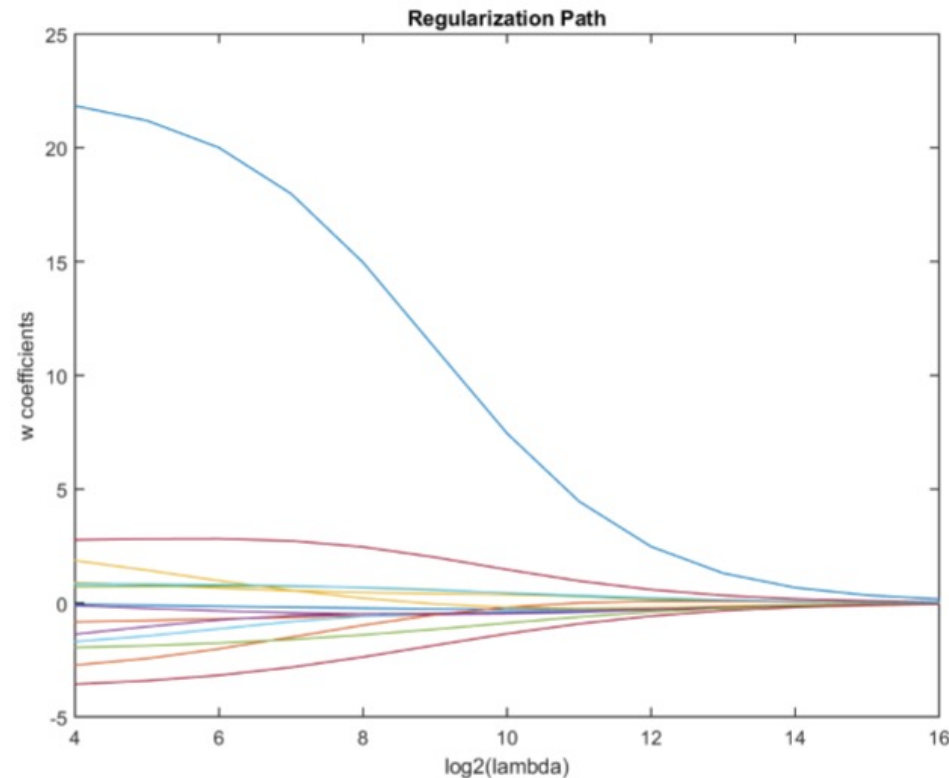
- Solution to a “least squares with L2-regularization” for different λ :

λ	w_1	w_2	w_3	w_4	w_5	$ Xw - y ^2$	$ w ^2$
0	-1.88	1.29	-2.63	1.78	-0.63	285.64	15.68
1	-1.88	1.28	-2.62	1.78	-0.64	285.64	15.62
4	-1.87	1.28	-2.59	1.77	-0.66	285.64	15.43
16	-1.84	1.27	-2.50	1.73	-0.73	285.71	14.76
64	-1.74	1.23	-2.22	1.59	-0.90	286.47	12.77
256	-1.43	1.08	-1.70	1.18	-1.05	292.60	8.60
1024	-0.87	0.73	-1.03	0.57	-0.81	321.29	3.33
4096	-0.35	0.31	-0.42	0.18	-0.36	374.27	0.56

- We get least squares with $\lambda = 0$.
 - But we can achieve similar training error with smaller $||w||$.
- $||Xw - y||$ increases with λ , and $||w||$ decreases with λ .
 - Though individual w_j can increase or decrease with λ .
 - Because we use the L2-norm, the large ones decrease the most.

Regularization Path

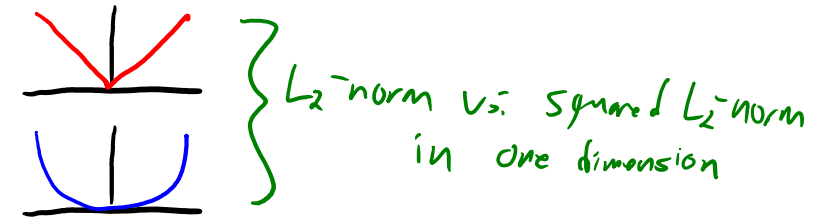
- **Regularization path** is a plot of the optimal weights ' w_j ' as ' λ ' varies:



- Starts with least squares with $\lambda = 0$, and w_j converge to 0 as λ grows.

Regularizing L2-norm vs. Squared L2-norm

- Why do we **regularize the square** of the L2-norm?
 - Why not regularize the L2-norm instead?
- **Regularization path is same** for L2-norm and squared L2-norm.
 - Though with different values of λ .
- But **squaring the L2-norm makes sure the gradient exists**.
 - The **L2-norm** is non-differentiable at $w=0$.
 - But the **squared L2-norm** is still differentiable at 0.



Solving L2-Regularized Least Squares Problem

- Solving for $\nabla f(w)=0$ to compute **L2-regularized** least squares:

– Objective:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

$$= \frac{1}{2} w^T X^T X w - w^T X^T y + \frac{1}{2} y^T y + \frac{\lambda}{2} w^T w \quad (\text{expand})$$

- Gradient:

$$\nabla f(w) = X^T X w - X^T y + 0 + \lambda w$$

- Setting gradient **equal to zero** vector:

$$X^T X w - X^T y + \lambda w = 0$$

$$X^T X w + \lambda w = X^T y \quad (\text{move terms with no 'w' to right})$$

- Factorize 'w' on the left side (identity matrix makes dimensions match):

$$(X^T X + \lambda I) w = X^T y$$

$$w = (X^T X + \lambda I)^{-1} X^T y$$

you can show that this matrix is always invertible.

Gradient Descent for L2-Regularized Least Squares

- The L2-regularized least squares objective and gradient:

$$f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2 \quad \nabla f(w) = X^T(Xw - y) + \lambda w$$

- Gradient descent iterations for L2-regularized least squares:

$$w^{t+1} = w^t - \alpha^t \left[\underbrace{X^T(Xw^t - y) + \lambda w^t}_{\nabla f(w^t)} \right]$$

- Cost of gradient descent iteration is still $O(nd)$.
 - Can show **number of iterations decrease as λ increases** (not obvious).
 - Do L2-regularized robust/brittle regression with smooth approximation.

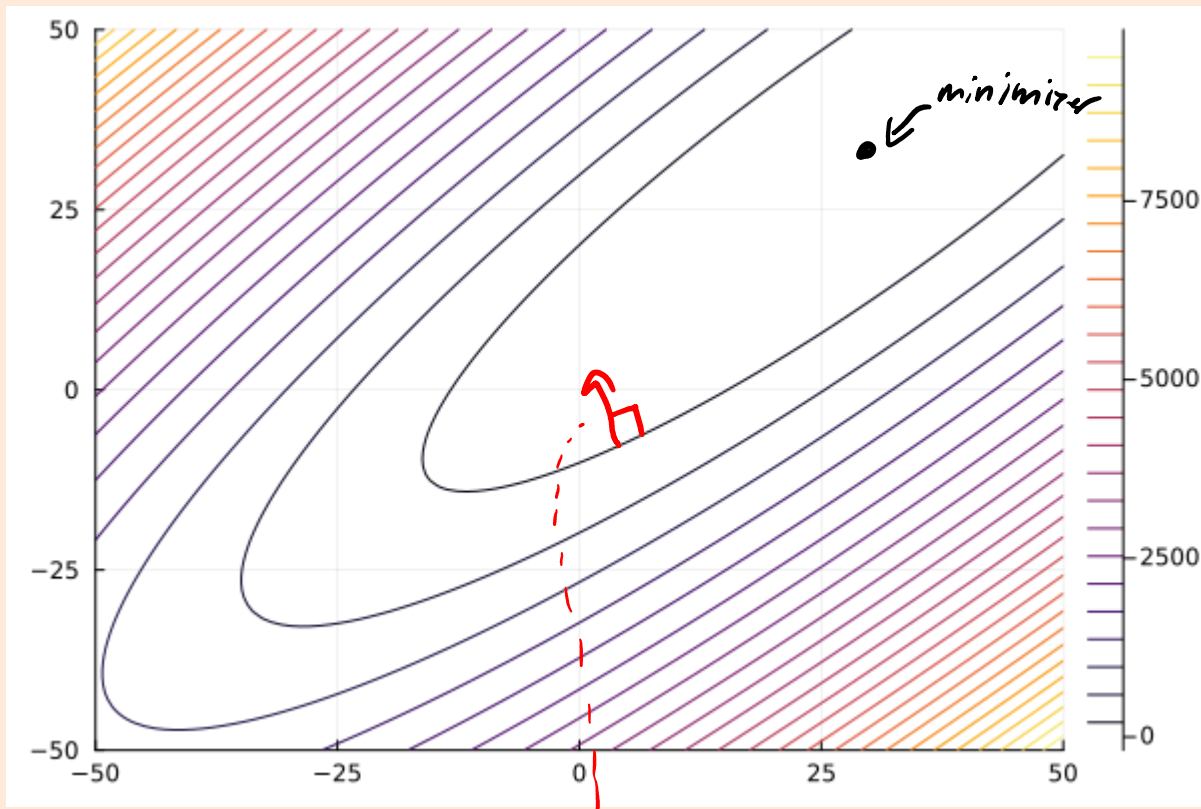
Why use L2-Regularization?

- It's a weird thing to do, but Mark says “always use regularization”.
 - “Almost always decreases test error” should already convince you.
- But here are 6 more reasons:
 1. Solution ‘w’ is **unique** (if training loss is convex).
 2. $X^T X$ does **not need to be invertible** (no collinearity issues).
 3. **Less sensitive** to changes in X or y.
 4. Gradient descent **converges faster** (bigger λ means fewer iterations).
 5. Stein's paradox: if $d \geq 3$, ‘shrinking’ **moves us closer to ‘true’ w**.
 6. Worst case: just set λ small and get the same performance.

Gradient descent converges faster?

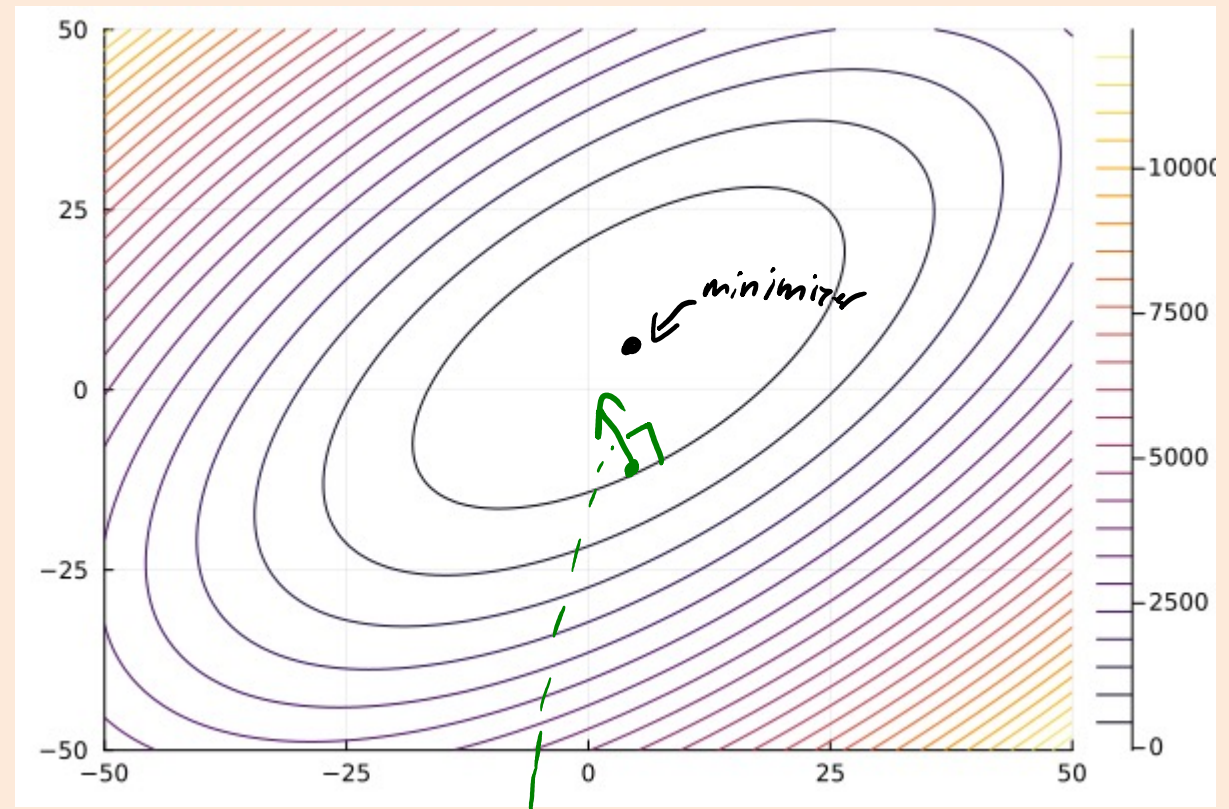
- With regularization, **gradient becomes a better direction** to follow:

Squared error (no regularization)



↓ gradient descent only
points slightly towards minimum

Squared error **plus L2-regularization**



↓ gradient descent points close
to minimizer

Next Topic: Standardizing Features

Features with Different Scales

- Consider continuous features with different scales:

Egg (#)	Milk (mL)	Fish (g)	Pasta (cups)
0	250	0	1
1	250	200	1
0	0	0	0.5
2	250	150	0

- Should we convert to some standard 'unit'?
 - It **doesn't matter for decision trees or naïve Bayes**.
 - They only look at one feature at a time.
 - It **does not matter for least squares**:
 - $w_j \cdot (100 \text{ mL})$ gives the same model as $w_j \cdot (0.1 \text{ L})$ with a different w_j .

Features with Different Scales

- Consider continuous features with different scales:

Egg (#)	Milk (mL)	Fish (g)	Pasta (cups)
0	250	0	1
1	250	200	1
0	0	0	0.5
2	250	150	0

- Should we convert to some standard 'unit'?
 - It **matters for k-nearest neighbours**:
 - "Distance" will be affected more by large features than small features.
 - It **matters for regularized least squares**:
 - Penalizing $(w_j)^2$ means different things if features 'j' are on different scales.

$$X = \begin{bmatrix} \vdots \\ \text{average of column } j \end{bmatrix}$$

- For each feature:

- Calculation:
- $$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$$
- $$\sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2}$$

- Replace x_{ij} with $\frac{x_{ij} - \mu_j}{\sigma_j}$

- Now measures “standard deviations from mean”.

- And changes in ' w_j ' have similar effect for any feature ' j '

- How should we **standardize test data**?

- **Wrong approach:** use mean and standard deviation of test data.

- Training and test mean and standard deviation might be very different.

- Right approach: use mean and standard deviation of training data.

- It is common to **standardize** continuous features:

- $$\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij} \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2}$$

2. Subtract mean and divide by standard deviation ("z-score")

Replace x_{ij} with $\frac{x_{ij} - \mu_j}{\sigma_j}$
 ons from mean.

- Now measures “standard deviation’s from mean”.
 - And changes in ‘ w_j ’ have similar effect for any feature ‘j’

- If we are doing 10-fold cross-validation:

- Compute μ_j and σ_j based on the 9 training folds (e.g., average over 9/10s of data).
 - Standardize the remaining (“validation”) fold with this “training” μ_j and σ_j .
 - Re-standardizing for different folds.
 - If you standardized before splitting, then validation fold would be influencing training.

Standardizing Target

- In regression, we sometimes **standardize the targets y_i** .
 - Puts targets on the same standard scale as standardized features:

$$\text{Replace } y_i \text{ with } \frac{y_i - \mu_y}{\sigma_y}$$

- With standardized target, setting $w = 0$ **predicts average y_i** :
 - High **regularization** makes us predict closer to the **average** value.
- Again, make sure you **standardize test data with the training stats**.
 - And **do not forget to “un-standardize” predictions** to get back to original space.
- Other common transformations of y_i are logarithm/exponent:

$$\text{Use } \log(y_i) \text{ or } \exp(\gamma y_i)$$

- Makes sense for geometric/exponential processes.

Summary

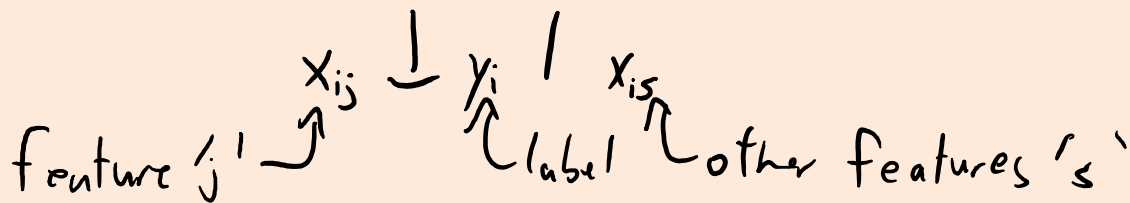
- “Relevance” is really hard to define.
 - Post-lecture bonus: “rough guide” to how different methods deal with this issue.
- Regularization:
 - Adding a penalty on model complexity.
- L2-regularization: penalty on L2-norm of regression weights ‘ w ’.
 - Trades training error against size of weights, almost always improves test error.
- Standardizing features:
 - For some models it makes sense to have features on the same scale.
- Next time: learning with an exponential number of irrelevant features.

Rough Guide to Feature Selection

Method\Issue	Dependence	Conditional Independence	Tiny effects	Context-Specific Relevance
Association (e.g., measure correlation between features 'j' and 'y')	Ok (takes "Parent" and "Parent2")	Bad (takes "Sibling", "Great-grandparent", etc.)	Ignores	Bad (misses features that must interact, "Partner" irrelevant given "Baby")
Regression Weight (fit least squares, take biggest $ w_j $)	Bad (can take irrelevant but collinear, can take none of "Parent1-3")	Ok (takes "Parent" not "Great-great-grandparent", if linear and 'n' large.	Ignores (unless collinear)	Ok (if linear, "Partner" relevant give "Baby")
Search and Score w/ Validation Error	Ok (takes at least one of "Parent" and "Parent2")	Bad (takes "Sibling", "Great-grandparent", etc.)	Allows (many false positives)	Ok (("Partner" relevant given "Baby")
Search and Score w/ L0-norm	Ok (takes exactly one of "Parent" and "Parent2")	Ok (takes "Parent" not "Great-grandparent" if linear-ish)	Ignores (even if collinear)	Ok (("Partner" relevant given "Baby")

Alternative to Search and Score: good old p-values

- **Hypothesis testing** (“constraint-based”) approach:
 - Generalization of the “association” approach to feature selection.
 - Performs a sequence of **conditional independence tests**.



“If I know features in 's' does feature 'j' tell me anything about label?”

- If they are independent (like “ $p < .05$ ”), say that ‘j’ is “irrelevant”.
- Common way to do the tests:
 - “Partial” correlation (numerical data).
 - “Conditional” mutual information (discrete data).

Testing-Based Feature Selection

- Hypothesis testing (“constraint-based”) approach:
- Too many possible tests, “greedy” method is for each ‘j’ do:

First test if $x_{ij} \perp y_i$

If still dependent test $x_{ij} \perp y_i \mid x_{iS}$ where ‘s’ has one feature

If still dependent test $x_{ij} \perp y_i \mid x_{iS}$ where ‘s’ now has two features dependence.

⋮

If still dependent when ‘s’ includes all other features, declare ‘j’ relevant.

Often choose features to minimize dependence.

- “Association approach” is the greedy method where you **only do the first test** (subsequent tests remove a lot of false positives).

Hypothesis-Based Feature Selection

- Advantages:
 - Deals with conditional independence.
 - Algorithm can **explain why it thinks 'j' is irrelevant**.
 - Doesn't necessarily need linearity.
- Disadvantages:
 - Deals badly with exact dependence: doesn't select "Parent" or "Parent2" if both present.
 - Usual warning about **testing multiple hypotheses**:
 - If you test $p < 0.05$ more than 20 times, you're going to make errors.
 - Greedy approach may be sub-optimal.
- Neither good nor bad:
 - Allows tiny effects.
 - Says "Parent" is irrelevant when you know "baby".
 - **This approach is sometimes better for finding relevant factors, not to select features for learning.**

Feature Selection Hierarchy

- Consider a linear models with **higher-order terms**,

$$\hat{y}_i = w_0 + w_1 x_{i1} + w_2 x_{i2} + w_3 x_{i3} + w_{12} x_{i1} x_{i2} + w_{13} x_{i1} x_{i3} + w_{23} x_{i2} x_{i3} + w_{123} x_{i1} x_{i2} x_{i3}$$

- The **number of higher-order terms may be too large**.
 - Can't even compute them all.
 - We need to somehow decide which terms we'll even consider.
- Consider the following **hierarchical constraint**:
 - You only **allow** $w_{12} \neq 0$ if $w_1 \neq 0$ and $w_2 \neq 0$.
 - "Only consider feature interaction if you are using both features already."

Hierarchical Forward Selection

- Hierarchical Forward Selection:
 - Usual forward selection, but **consider interaction terms obeying hierarchy**.
 - Only consider $w_{12} \neq 0$ once $w_1 \neq 0$ and $w_2 \neq 0$.
 - Only allow $w_{123} \neq 0$ once $w_{12} \neq 0$ and $w_{13} \neq 0$ and $w_{23} \neq 0$.
 - Only allow $w_{1234} \neq 0$ once all threeway interactions are present.

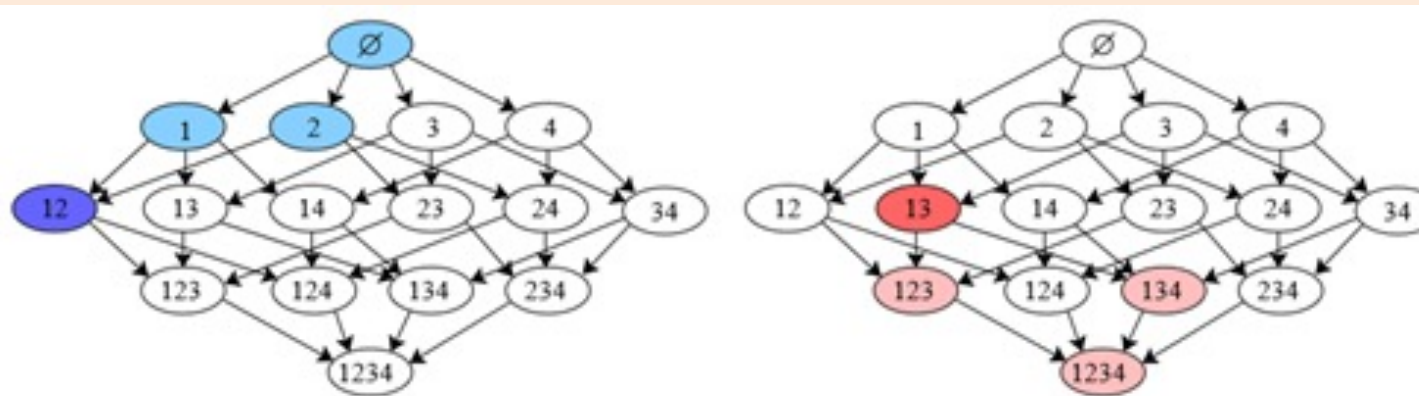


Fig 9: Power set of the set $\{1, \dots, 4\}$: in blue, an authorized set of selected subsets. In red, an example of a group used within the norm (a subset and all of its descendants in the DAG).

Interventional Data

- The difference between **observational** and **interventional** data:
 - If I **see** that my watch says 10:45, class is almost over (**observational**).
 - If I **set** my watch to say 10:45, it doesn't help (**interventional**).
- The **intervention** can help discover causal effects:
 - “Watch” is only predictive of “time” in observational setting (so not causal).
- General idea for **identifying causal effects**:
 - “Force” the variable to take a certain value, then measure the effect.
 - If the dependency remains, there is a causal effect.
 - We “break” connections from reverse causality, common effects, or confounding.

Causality and Dataset Collection

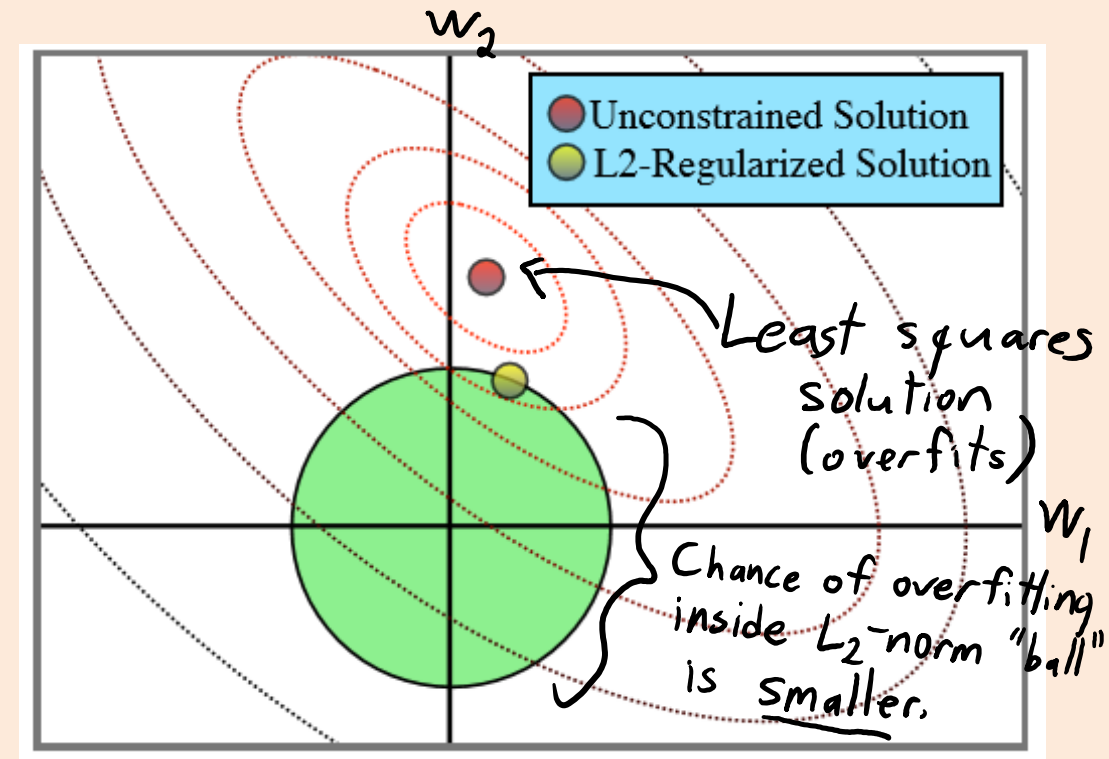
- This has to do with the **way you collect data**:
 - You **can't "look" for variables taking the value** "after the fact".
 - You **need to manipulate the value of the variable**, then watch for changes.
- This is the basis for **randomized control trial** in medicine:
 - Randomly assigning pills "forces" value of "treatment" variable.
 - Randomization means they aren't taking the pill due to confounding factors.
 - Differences between people who did and did not take pill should be caused by pill.
 - Include a "control" as a value to prevent placebo effect as confounding.
- See also Simpson's Paradox:
 - <https://www.youtube.com/watch?v=ebEkn-BiW5k>

L2-Regularization

- Standard **regularization** strategy is **L2-regularization**:

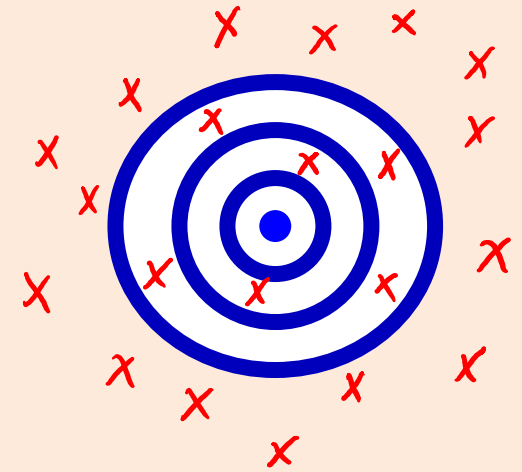
$$f(w) = \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \quad \text{or} \quad f(w) = \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2$$

- Equivalent to minimizing squared error but keeping L2-norm small.



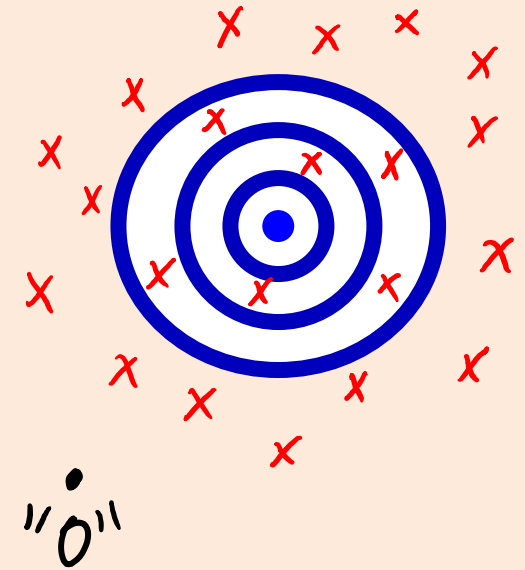
Regularization/Shrinking Paradox

- We throw darts at a target:
 - Assume we don't always hit the exact center.
 - Assume the darts follow a symmetric pattern around center.



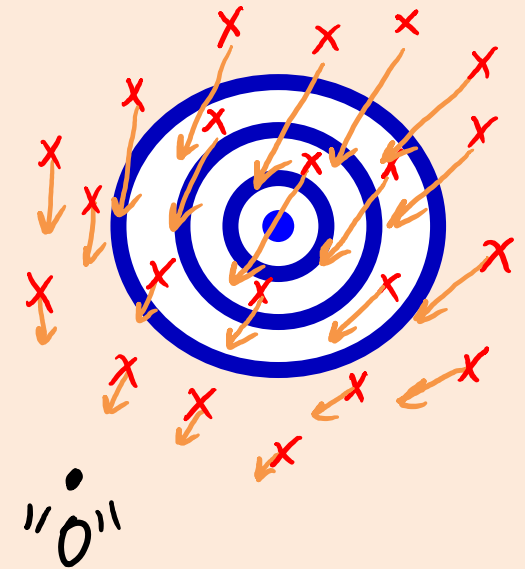
Regularization/Shrinking Paradox

- We throw darts at a target:
 - Assume we don't always hit the exact center.
 - Assume the darts follow a symmetric pattern around center.
- Shrinkage of the darts :
 1. Choose some **arbitrary** location '0'.
 2. Measure distances from darts to '0'.



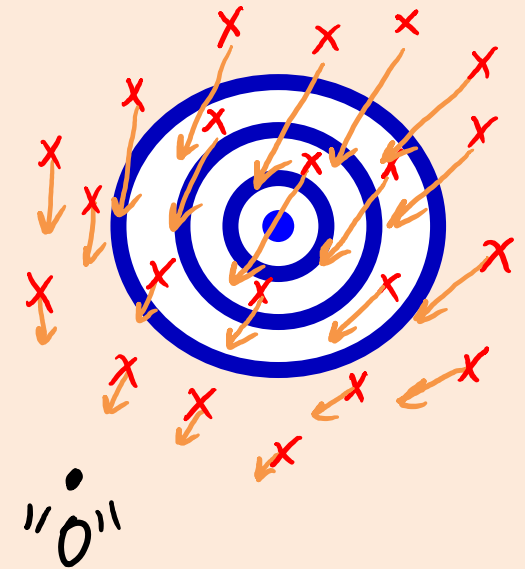
Regularization/Shrinking Paradox

- We throw darts at a target:
 - Assume we don't always hit the exact center.
 - Assume the darts follow a symmetric pattern around center.
- Shrinkage of the darts :
 1. Choose some **arbitrary** location '0'.
 2. Measure distances from darts to '0'.
 3. **Move misses towards '0', by *small* amount proportional to distance from 0.**
- If small enough, **darts will be closer to center on average.**



Regularization/Shrinking Paradox

- We throw darts at a target:
 - Assume we don't always hit the exact center.
 - Assume the darts follow a symmetric pattern around center.
- Shrinkage of the darts :
 1. Choose some **arbitrary** location '0'.
 2. Measure distances from darts to '0'.
 3. **Move misses towards '0'**, by *small* amount **proportional to distance from 0**.
- If small enough, **darts will be closer to center on average**.



Visualization of the related higher-dimensional paradox that the mean of data coming from a Gaussian is not the best estimate of the mean of the Gaussian in 3-dimensions or higher: <https://www.naftaliharris.com/blog/steinviz>